

# 10

## Aggregating and Visualizing Statistics Using MongoDB and Victory

In this chapter, we are going to learn how to collect, aggregate, and visualize statistics for our blog application using MongoDB and Victory. We start out by learning how we can collect events from users viewing blog posts. Then, we randomly generate some events to have a dataset to work with. We use this dataset to learn how to aggregate data with MongoDB and generate summary statistics, such as the number of views per post, or the average session duration on a post. This kind of information will help authors know how well their posts are doing. Finally, we create some graphs to visualize these aggregated statistics using the Victory library.

In this chapter, we are going to cover the following main topics:

- Collecting and simulating events
- Aggregating data with MongoDB
- Implementing data aggregation in the backend
- Integrating and visualizing data on the frontend using Victory

## Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing for Full-Stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain

steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1* and *2*.

You can find the code for this chapter on GitHub:

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch10>.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for this chapter can be found at:

[https://youtu.be/DmSq2P\\_IQQs](https://youtu.be/DmSq2P_IQQs).

## Collecting and simulating events

Before we can get started aggregating and visualizing statistics, we first need to collect (and later simulate) events, which we are going to use to create the statistics. We will start by thinking about which data we want to collect, and which data would be useful for us. We will focus on post views for now, so we would like to show the following statistics per post:

- Total number of views on a post
- Daily views on a post
- Daily average viewing duration on a post

Let's start by creating the database model for events that will allow us to show these statistics.

### Creating the event model

To create these statistics, we need to collect events from users. Events will contain a reference to a post, a session ID to track events from the same viewing, an action (started viewing, ended viewing), and a date of when the event happened.

Let's get started defining the database model for events:

1. Copy the existing **ch9** folder to a new **ch10** folder, as follows:

```
$ cp -R ch9 ch10
```

2. Open the **ch10** folder in VS Code.
3. Create a new **backend/src/db/models/event.js** file. Inside this file, define a schema that contains a reference to a post:

```
import mongoose, { Schema } from 'mongoose'
const eventsSchema = new Schema(
  {
    post: { type: Schema.Types.ObjectId, ref: 'post', required: true },
```

4. Then define a **session**, **action**, and **date**:

```
    session: { type: String, required: true },
    action: { type: String, required: true },
    date: { type: Date, required: true },
  },
  { timestamps: true },
)
```

5. Finally, export the model:

```
export const Event = mongoose.model('events', eventsSchema)
```

Now that we have defined the database model, let's continue by defining a service function and route to track events.

## Defining a service function and route to track events

Now that we have successfully defined our database model for events, let's create a service function and route to track new events, as follows:

1. To generate session IDs, we are going to use the **uuid** library, which generates **universally unique identifiers (UUIDs)** for us. Install it by running the following commands:

```
$ cd backend/
```

```
$ npm install uuid@9.0.1
```

2. Create a new **backend/src/services/events.js** file. Inside it, import the **v4** function from **uuid** and the **Event** model and define a function to create a new event document, as follows:

```
import { v4 as uuidv4 } from 'uuid'
import { Event } from '../db/models/event.js'
export async function trackEvent({
  postId,
  action,
  session = uuidv4(),
  date = Date.now(),
}) {
  const event = new Event({ post: postId, action, session, date })
  return await event.save()
}
```

In the arguments to the function, we set the default session ID to a randomly generated UUID and the date to the current date.

3. Create a new **backend/src/routes/events.js** file. Inside it, import the **trackEvent** function and the **getPostById** function:

```
import { trackEvent } from '../services/events.js'
import { getPostById } from '../services/posts.js'
```

4. Define a new **POST /api/v1/events** route, in which we get the **postId**, **session**, and **action** from the body:

```
export function eventRoutes(app) {
  app.post('/api/v1/events', async (req, res) => {
    try {
      const { postId, session, action } = req.body
```

5. Then, we check whether a post with the given ID exists in the database. If not, we return a **400 Bad Request** status code:

```
const post = await getPostById(postId)
if (post === null) return res.status(400).end()
```

6. If the post exists, we get the session ID and use the **trackEvent** function to create a new event:

```

    const event = await trackEvent({ postId, session, action })
    return res.json({ session: event.session })
  } catch (err) {
    console.error('error tracking action', err)
    return res.status(500).end()
  }
})
}

```

7. Edit **backend/src/app.js** and import **eventRoutes**:

```
import { eventRoutes } from './routes/events.js'
```

8. Then mount the routes to the app:

```

postRoutes(app)
userRoutes(app)
eventRoutes(app)

```

9. Start the backend as follows (and keep it running for future development):

```

$ cd backend/
$ npm run dev

```

Now that we have successfully defined a backend route to track events, let's implement event collection on the frontend.

## Collecting events on the frontend

After defining the route, let's create the API function for the frontend and define a way to track when a user started and ended viewing a post.

Follow these steps to collect events on the frontend:

1. Create a new **src/api/events.js** file and define a **postTrackEvent** function, which takes an event object and sends it to the previously defined route:

```

export const postTrackEvent = (event) =>
  fetch(`${import.meta.env.VITE_BACKEND_URL}/events`, {

```

```

method: 'POST',
headers: {
  'Content-Type': 'application/json',
},
body: JSON.stringify(event),
}).then((res) => res.json())

```

2. Edit **src/pages/ViewPost.jsx** and import the **useEffect**, **useState**, and **useMutation** hooks:

```

import { useEffect, useState } from 'react'
import { useQuery, useMutation } from '@tanstack/react-query'

```

3. Additionally, import the **postTrackEvent** API function:

```

import { postTrackEvent } from '../api/events.js'

```

4. Now, inside the **ViewPost** function, define a new state hook to store the session ID, and a mutation to track the events. When an event is successfully tracked, we get a session ID from the backend. We store this in the state hook:

```

const [session, setSession] = useState()
const trackEventMutation = useMutation({
  mutationFn: (action) => postTrackEvent({ postId, action, session }),
  onSuccess: (data) => setSession(data?.session),
})

```

5. Then, define a new effect hook in which we track a **startView** event a second after the user opened the post (to prevent tracking accidental events, such as from quick refreshes), and an **endView** event when the effect hook unmounts. We give it no dependencies to ensure the effect hook is only triggered when the page mounts and unmounts:

```

useEffect(() => {
  let timeout = setTimeout(() => {
    trackEventMutation.mutate('startView')
    timeout = null
  }, 1000)
  return () => {
    if (timeout) clearTimeout(timeout)
    else trackEventMutation.mutate('endView')
  }
}, [])

```

```
}  
}, [])
```

6. Start the frontend as follows (and keep it running for future development):

```
$ npm run dev
```

Make sure to run this command in the root of the **ch10** folder, not inside the **backend** folder.

If you now open a post in your browser and take a look at the **Network** tab of the inspector, you will see that after a second, the **startView** event is tracked. When we navigate away from the page, the **endView** event is tracked.

Let's now move on to simulating events so that we have more data to aggregate and visualize later.

## Simulating events

Simulating events is a great way to generate sample data to be used for testing the aggregations and visualizations. In our simulation, we first clear all current users from the database and then create a set of sample users. We repeat the same process for posts, and then for events, simulating that a random user creates a post and that someone views a random post for a random amount of time.

Follow these steps to implement a simulation:

1. First of all, we should change the database to avoid losing any data we previously created in the other chapters. Edit **backend/.env** and change the following line from **blog** to **blog-simulated**:

```
DATABASE_URL=mongodb://localhost:27017/blog-simulated
```

2. Now, create a new **backend/simulateEvents.js** file, in which we import **dotenv**, the **initDatabase** function, and all the relevant models and service functions:

```
import dotenv from 'dotenv'
```

```

dotenv.config()
import { initDatabase } from './src/db/init.js'
import { Post } from './src/db/models/post.js'
import { User } from './src/db/models/user.js'
import { Event } from './src/db/models/event.js'
import { createUser } from './src/services/users.js'
import { createPost } from './src/services/posts.js'
import { trackEvent } from './src/services/events.js'

```

3. Define a start time for the simulation, which here is set to 30 days ago (30 days \* 24 hours \* 60 minutes \* 60 seconds \* 1000 milliseconds), and an end time, which is now:

```

const simulationStart = Date.now() - 1000 * 60 * 60 * 24 * 30
const simulationEnd = Date.now()

```

4. We also define the number of users, posts, and views to simulate:

```

const simulatedUsers = 5
const simulatedPosts = 10
const simulatedViews = 10000

```

5. Then, define the **simulateEvents** function, in which we first initialize the database:

```

async function simulateEvents() {
  const connection = await initDatabase()

```

6. Next, *delete* all existing users and create new users by initializing an empty array with the number of users to be simulated and mapping over it:

```

await User.deleteMany({})
const createdUsers = await Promise.all(
  Array(simulatedUsers)
    .fill(null)
    .map(
      async (_, u) =>
        await createUser({
          username: `user-${u}`,
          password: `password-${u}`,
        }),
    ),
)

```



```
)  
console.log(`created ${createdUsers.length} users`)
```

### INFO

*The **Array(X)** function can be used to create an array with **X** entries, which then needs to be filled with an initial value before it can be iterated over.*

7. Now, repeat the same process for posts:

```
await Post.deleteMany({})  
const createdPosts = await Promise.all(  
  Array(simulatedPosts)  
    .fill(null)  
    .map(async (_, p) => {  
      const randomUser =  
        createdUsers[Math.floor(Math.random() * simulatedUsers)]  
      return await createPost(randomUser._id, {  
        title: `Test Post ${p}`,  
        contents: `This is a test post ${p}`,  
      })  
    })  
  ),  
)  
console.log(`created ${createdPosts.length} posts`)
```

### INFO

*We use **Math.floor(Math.random() \* maxNumber)** to create a random integer between 0 and maxNumber (non-inclusive), which is perfect to be used for indexing an array.*

8. Lastly, we repeat the same for events:

```
await Event.deleteMany({})  
const createdViews = await Promise.all(  
  Array(simulatedViews)  
    .fill(null)  
    .map(async () => {  
      const randomPost =  
        createdPosts[Math.floor(Math.random() * simulatedPosts)]
```

9. Here, we start the session at a random time within the defined simulation dates:

```
const sessionStart =  
  simulationStart + Math.random() * (simulationEnd - simulationStart)
```

10. And we end it randomly after 0 to 5 minutes:

```
const sessionEnd =  
  sessionStart + 1000 * Math.floor(Math.random() * 60 * 5)
```

11. Now, we simulate the event collection, first by creating a **startView** event:

```
const event = await trackEvent({  
  postId: randomPost._id,  
  action: 'startView',  
  date: new Date(sessionStart),  
})
```

12. And then we simulate an **endView** event, where we use the session ID returned from the first event:

```
    await trackEvent({  
      postId: randomPost._id,  
      session: event.session,  
      action: 'endView',  
      date: new Date(sessionEnd),  
    })  
  }),  
)  
console.log(`successfully simulated ${createdViews.length} views`)
```

13. Lastly, we disconnect from the database and call the function:

```
    await connection.disconnect()  
  }  
  simulateEvents()
```

14. Our simulation is now ready to be used! Execute the following command to start it:

```
$ cd backend/
```

```
$ node simulateEvents.js
```

You will see that the simulation first creates 5 users, then 10 posts, and finally simulates 10,000 views.

In the next section, we are going to use this dataset to try out some aggregations with MongoDB!

## Aggregating data with MongoDB

Sometimes, we do not just want to simply retrieve data from the database, but instead, we want to create some statistics from the data by combining and summarizing it. This process is called **data aggregation**, and it can help us understand more about the data. For example, we can count the total number of views per post, get the number of daily views per post, or calculate the average session duration when viewing a post.

MongoDB supports a special aggregation syntax using the `.aggregate()` function on a collection. Using this aggregation functionality from MongoDB allows us to efficiently query and process documents. The operations it provides are similar to what can be done with **Structured Query Language (SQL)** queries. Mainly, we are going to use the following aggregation operations:

- **\$match**: Used to filter documents
- **\$group**: Used to group documents by a certain property
- **\$project**: Used to map properties to different properties, or process them
- **\$sort**: Used to sort documents

### INFO

*MongoDB provides many more advanced aggregation operations, all of which can be found in their documentation*

*(<https://www.mongodb.com/docs/manual/aggregation/>). They are also constantly adding more operations to make aggregation even more powerful.*

The **aggregate** function works by providing an array of objects, each of which defines a **stage** of the **aggregation pipeline**. We are going to learn more about aggregations in this chapter by using them in practice.

## Getting the total number of views per post

The first aggregation that we are going to define is a way to get the total number of views per post. For such an aggregation, we are going to need **\$match** to filter all **startView** actions (otherwise we would be counting views twice, because there is also an **endView** action for each blog post view), and **\$group** to group the results by post ID and then return the number of documents using **\$count**.

Follow these steps to create your first aggregation pipeline:

1. Create a new **backend/playground/** folder for our playground scripts.
2. Click on the MongoDB extension (the leaf icon) in the VS Code sidebar.
3. Connect to the database, then expand the **Playgrounds** section (if it is not expanded already), and click on the **Create New Playground** button.

A new file will open up with some code already predefined for us.

*Delete* all predefined code, as we are going to *replace* it with our own.

4. First, define the **use** and **db** globals, which the MongoDB Playground provides for us:

```
/* global use, db */
```

5. Then, use the **blog-simulated** database:

```
use('blog-simulated')
```

6. Now, execute the following aggregation function:

```
db.getCollection('events').aggregate([
```

7. The first stage of the pipeline will be matching all **startView** actions:

```
{  
  $match: { action: 'startView' },  
},
```

8. Then, we group by post. The **\$group** stage requires us to define an **\_id**, which contains the property to be grouped by. We need to use the **\$** operator to resolve the variable to be used, so **\$post** will access the **event.post** property (which contains a post ID):

```
{
  $group: {
    _id: '$post',
    views: { $count: {} },
  },
},
])
```

9. Save the script as a **backend/playground/views-per-post.mongoddb.js** file.
10. Click on the **Play** icon at the top right to run the script. A new tab will open with the results of the aggregation:

```
JS views-per-post.mongodb.js {} Playground Result X
1  [
2    {
3      "_id": {
4        "$oid": "64d294acb1fac3e6f41f5f1d"
5      },
6      "views": 994
7    },
8    {
9      "_id": {
10       "$oid": "64d294acb1fac3e6f41f5f15"
11     },
12     "views": 1005
13   },
14   {
15     "_id": {
16       "$oid": "64d294acb1fac3e6f41f5f1a"
17     },
18     "views": 971
19   },
20   {
21     "_id": {
22       "$oid": "64d294acb1fac3e6f41f5f1c"
23     },
24     "views": 1027
25   },
26 ]
```

Figure 10.1 – Our first MongoDB aggregation result!

After creating and executing our first simple aggregation, let's continue practicing by writing more advanced aggregations.

## Getting the number of daily views per post

Now that we are already familiar with the general process of writing MongoDB aggregations, let's try writing a bit more complicated aggregation: getting the number of daily views per post. Follow these steps to create it:

1. Create a new playground file, as before, with the following aggregation function:

```
/* global use, db */
use('blog-simulated')
db.getCollection('events').aggregate([
```

2. Again, we first match only the **startView** actions:

```
{
  $match: { action: 'startView' },
},
```

3. Then we use **\$project** to keep the **post** property, and define a new **day** property, which uses the **\$dateTrunc** function to simplify the **date** property to only cover days (instead of containing the full timestamp):

```
{
  $project: {
    post: '$post',
    day: { $dateTrunc: { date: '$date', unit: 'day' } },
  },
},
```

An important thing to keep in mind with **\$project** is that only properties that are listed here will be passed on to further stages in the pipeline, so we need to list all properties that we are still going to need later here!

4. Finally, we use **\$group** to group the documents by **post** and **day** by passing an object to the **\_id** property. We use **\$count** again to count the number of documents in each group:

```
{
  $group: {
    _id: { post: '$post', day: '$day' },
    views: { $count: {} },
  },
},
])
```

5. Save the script as a **backend/playground/views-per-post-per-day.mongoddb.js** file.
6. Run this script by clicking on the **Play** button and you will see that we are now getting a list of documents grouped by post and day, and the

corresponding number of views of a certain post on a certain day:

```
{} Playground Result ×
1  [
2    {
3      "_id": {
4        "post": {
5          "$oid": "65c8ef3599c4fdd0c066ed74"
6        },
7        "day": {
8          "$date": "2024-01-30T00:00:00Z"
9        }
10     },
11     "views": 36
12   },
13   {
14     "_id": {
15       "post": {
16         "$oid": "65c8ef3599c4fdd0c066ed72"
17       },
18       "day": {
19         "$date": "2024-01-12T00:00:00Z"
20       }
21     },
22     "views": 7
23   },
```

Figure 10.2 – Showing the number of views per post per day

After getting the number of daily views per post, let's continue practicing by calculating the average session duration.

## Calculating the average session duration

As you may remember, we are first sending a **startView** action, and then later an **endView** action, both of which have a separate **date**. Let's use aggregations to group these two actions together into a single document, and then compute the duration of a session:

1. Create a new playground file, and start writing an aggregation that first creates some new properties using **\$project**, and keeps the **session** property, as we will need it later:



```

/* global use, db */
use('blog-simulated')
db.getCollection('events').aggregate([
  {
    $project: {
      session: '$session',
      startDate: {
        $cond: [{ $eq: ['$action', 'startView'] }, '$date', undefined],
      },
      endDate: { $cond: [{ $eq: ['$action', 'endView'] }, '$date', undefined]
    },
  },
],

```

Here, we are using the **\$cond** operator to create a conditional (kind of like a ternary/if statement). It accepts an array with three elements: the first being a condition, the next a result if the condition matches, and lastly, a result if the condition does not match. In our case, we check whether the **action** property is **startView** (using the **\$eq** operator). If true, then we set the date to the **startDate** property. Otherwise, we do not define the **startDate** property. Similarly, if the action is **endView**, we create an **endDate** property.

2. Now, we can group the documents by the session ID and select the lowest start date and the highest end date of a session:

```

{
  $group: {
    _id: '$session',
    startDate: { $min: '$startDate' },
    endDate: { $max: '$endDate' },
  },
},

```

There should only be one **startView** and **endView** action per session anyway, but we cannot guarantee this, so we need to aggregate them down into a single value!

3. Finally, we use **\$project** again to rename the **\_id** property to **session**, and calculate the **duration** by subtracting the **startDate** from the **endDate**:

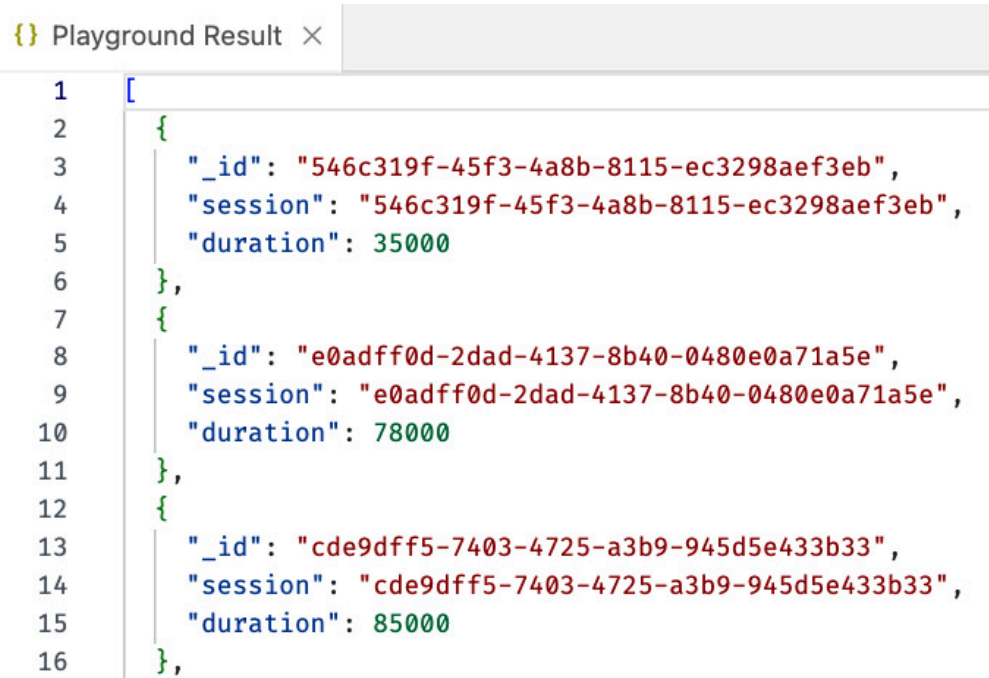
```

{
  $project: {

```

```
    session: '$_id',
    duration: { $subtract: ['$endDate', '$startDate'] },
  },
},
])
```

4. Save the script as a **backend/playground/session-duration.mongodb.js** file.
5. Run the script and you will see a list of documents with a session ID and a corresponding duration in milliseconds:



The screenshot shows a 'Playground Result' window with a close button. It displays an array of three documents. Each document has three fields: '\_id' (a long hexadecimal string), 'session' (the same hexadecimal string as the \_id), and 'duration' (a numerical value in milliseconds). The documents are separated by commas. The first document has a duration of 35000, the second 78000, and the third 85000.

```
1 [
2   {
3     "_id": "546c319f-45f3-4a8b-8115-ec3298aef3eb",
4     "session": "546c319f-45f3-4a8b-8115-ec3298aef3eb",
5     "duration": 35000
6   },
7   {
8     "_id": "e0adff0d-2dad-4137-8b40-0480e0a71a5e",
9     "session": "e0adff0d-2dad-4137-8b40-0480e0a71a5e",
10    "duration": 78000
11  },
12  {
13    "_id": "cde9dff5-7403-4725-a3b9-945d5e433b33",
14    "session": "cde9dff5-7403-4725-a3b9-945d5e433b33",
15    "duration": 85000
16  },
17 ]
```

Figure 10.3 – Aggregation result of the session durations

Now that we are more familiar with how data aggregation works in MongoDB, let's implement similar aggregations in our backend!

## Implementing data aggregation in the backend

For our backend, we are going to use very similar aggregation pipelines. However, we need to adjust them slightly, as we always want to get the data for a single post only. As such, we will first be using **\$match** to filter our documents. This also ensures that the aggregation stays fast, even if we have millions of events in our database, because we are first filtering down to all events of a single post!

## Defining aggregation service functions

Follow these steps to implement the aggregation functions in the backend:

1. Edit **backend/src/services/events.js** and define a new function to get the total number of views for a post. In this case, we can simplify our code by using the **countDocuments** function instead of the aggregate function:

```
export async function getTotalViews(postId) {
  return {
    views: await Event.countDocuments({ post: postId, action: 'startView' }),
  }
}
```

2. Next, define a new function to get the daily views of a post with a given ID. We now use the **\$match** operation to only get the **startView** actions of a certain post:

```
export async function getDailyViews(postId) {
  return await Event.aggregate([
    {
      $match: {
        post: postId,
        action: 'startView',
      },
    },
  ],
```

3. Then, we use the **\$group** operation in combination with **\$dateTrunc** to get the views per day, just like we did before in the MongoDB Playground script:

```
{
  $group: {
    _id: {
      $dateTrunc: { date: '$date', unit: 'day' },
    },
    views: { $count: {} },
  },
},
```

4. Lastly, we use the **\$sort** operation to sort the resulting documents by **\_id** (which contains the day):

```
{
  $sort: { _id: 1 },
},
])
}
```

5. For the last function, we use our session duration aggregation, but extend it a little bit to give the average duration per day. Again, we first need to match a post ID:

```
export async function getDailyDurations(postId) {
  return await Event.aggregate([
    {
      $match: {
        post: postId,
      },
    },
  ],
```

6. Then, we use the same **\$project** and **\$group** operations to get the **session**, **startDate**, and **endDate**, just like we did before:

```
{
  $project: {
    session: '$session',
    startDate: {
      $cond: [{ $eq: ['$action', 'startView'] }, '$date', undefined],
    },
    endDate: {
      $cond: [{ $eq: ['$action', 'endView'] }, '$date', undefined],
    },
  },
},
{
  $group: {
    _id: '$session',
    startDate: { $min: '$startDate' },
    endDate: { $max: '$endDate' },
  },
},
},
```

7. Now, we use the **\$project** operation to get **day** from our **startDate**, like we did in the previous aggregation where we got the number of daily views of a post:

```
{
  $project: {
    day: { $dateTrunc: { date: '$startDate', unit: 'day' } },
    duration: { $subtract: ['$endDate', '$startDate'] },
  },
},
```

8. We group the results per day, and calculate the average duration of a day:

```
{
  $group: {
    _id: '$day',
    averageDuration: { $avg: '$duration' },
  },
},
```

9. Finally, we sort the results per day:

```
{
  $sort: { _id: 1 },
},
])
}
```

As we can see, aggregation pipelines are extremely powerful and allow us to do a lot of data processing directly in the database! In the next section, we are going to create routes for these aggregation functions.

## Defining the routes

Defining the routes is pretty straightforward; we simply check whether a post with the given ID exists, and if it does, we return the results from the corresponding aggregation service function. Let's start defining the routes:

1. Edit **backend/src/routes/events.js** and import the **getTotalViews**, **getDailyViews**, and **getDailyDurations** functions:

```
import {
  trackEvent,
  getTotalViews,
  getDailyViews,
  getDailyDurations,
} from '../services/events.js'
```

2. Next, inside the **eventRoutes** function, define a new route for getting the total number of views of a post, as follows:

```
app.get('/api/v1/events/totalViews/:postId', async (req, res) => {
  try {
    const { postId } = req.params
    const post = await getPostById(postId)
    if (post === null) return res.status(400).end()
    const stats = await getTotalViews(post._id)
    return res.json(stats)
  } catch (err) {
    console.error('error getting stats', err)
    return res.status(500).end()
  }
})
```

3. Then define a similar route for the number of daily views of a post:

```
app.get('/api/v1/events/dailyViews/:postId', async (req, res) => {
  try {
    const { postId } = req.params
    const post = await getPostById(postId)
    if (post === null) return res.status(400).end()
    const stats = await getDailyViews(post._id)
    return res.json(stats)
  } catch (err) {
    console.error('error getting stats', err)
    return res.status(500).end()
  }
})
```

4. And finally, define a route for the daily average viewing duration of a post:

```
app.get('/api/v1/events/dailyDurations/:postId', async (req, res) => {
  try {
    const { postId } = req.params
    const post = await getPostById(postId)
    if (post === null) return res.status(400).end()
    const stats = await getDailyDurations(post._id)
    return res.json(stats)
  } catch (err) {
    console.error('error getting stats', err)
    return res.status(500).end()
  }
})
```

Now that we have successfully defined routes for our aggregation functions, it's time to integrate them into the frontend and start visualizing the data we have been simulating and collecting!

## Integrating and visualizing data on the frontend using Victory

In this final section, we are going to integrate the aggregation endpoints we previously defined. Then, we are going to introduce the Victory library in the frontend to create graphs to visualize our aggregated data!

### Integrating the aggregation API

First of all, we need to integrate the API routes in the frontend, as follows:

1. Edit the **src/api/events.js** file and add three new API functions to get the total views, daily views, and daily durations of a post:

```
export const getTotalViews = (postId) =>
  fetch(`${import.meta.env.VITE_BACKEND_URL}/events/totalViews/${postId}`).then
    (res) => res.json(),
  )
export const getDailyViews = (postId) =>
  fetch(`${import.meta.env.VITE_BACKEND_URL}/events/dailyViews/${postId}`).then
    (res) => res.json(),
  )
```

```
export const getDailyDurations = (postId) =>
  fetch(
    `${import.meta.env.VITE_BACKEND_URL}/events/dailyDurations/${postId}`,
  ).then((res) => res.json())
```

2. Create a new **src/components/PostStats.jsx** file, in which we are going to query these new API routes. Start by importing **useQuery**, **PropTypes**, and the three API functions:

```
import { useQuery } from '@tanstack/react-query'
import PropTypes from 'prop-types'
import {
  getTotalViews,
  getDailyViews,
  getDailyDurations,
} from '../api/events.js'
```

3. Define a new component that takes a **postId** and fetches all the stats that we aggregated on the backend using query hooks:

```
export function PostStats({ postId }) {
  const totalViews = useQuery({
    queryKey: ['totalViews', postId],
    queryFn: () => getTotalViews(postId),
  })
  const dailyViews = useQuery({
    queryKey: ['dailyViews', postId],
    queryFn: () => getDailyViews(postId),
  })
  const dailyDurations = useQuery({
    queryKey: ['dailyDurations', postId],
    queryFn: () => getDailyDurations(postId),
  })
}
```

4. While the stats are loading, we display a simple loading message:

```
if (
  totalViews.isLoading ||
  dailyViews.isLoading ||
  dailyDurations.isLoading
) {
  return <div>loading stats...</div>
}
```



5. Once the stats are finished loading, we can display them. For now, we simply display the total number of views and the JSON responses from the other two API requests:

```
return (
  <div>
    <b>{totalViews.data?.views} total views</b>
    <pre>{JSON.stringify(dailyViews.data)}</pre>
    <pre>{JSON.stringify(dailyDurations.data)}</pre>
  </div>
)
```

6. We still need to define the prop types for this component, as follows:

```
PostStats.propTypes = {
  postId: PropTypes.string.isRequired,
}
```

7. Now we can render the **PostStats** component in our **ViewPost** page component. Edit **src/pages/ViewPost.jsx** and import the **PostStats** component there:

```
import { PostStats } from '../components/PostStats.jsx'
```

8. Then, at the bottom of the component, render the stats as follows:

```
{post ? (
  <div>
    <Post {...post} fullPost />
    <hr />
    <PostStats postId={postId} />
  </div>
) : (
  `Post with id ${postId} not found.`
)}
```

If you open a post on the frontend now (you may need to refresh the frontend if you see an error), you will see that all the stats are properly

fetches! Now, all that's left to do is visualize the daily stats using Victory!

## Visualizing data using Victory

Victory is a React library that provides modular components that can be used to create charts and all kinds of data visualizations. It even supports interactive visualization tools, such as brushing and grouping (where you, for example, select a certain section of a graph to more closely inspect it on other graphs). In this chapter, we are only going to scratch the surface of what Victory can do, as data visualization in React is a big topic on its own.

You can find more information about Victory on their official website:

<https://commerce.nearform.com/open-source/victory/>

### Creating a bar chart

Let's get started visualizing our data using Victory now:

1. Install the library by executing the following command in the root of the project:

```
$ npm install victory@36.9.1
```

2. Edit **src/components/PostStats.jsx** and import the following components from Victory:

```
import {  
  VictoryChart,  
  VictoryTooltip,  
  VictoryBar,  
  VictoryLine,  
  VictoryVoronoiContainer,  
} from 'victory'
```

3. Replace the **<pre>** tags at the end of the component with the following charts, starting with the daily views chart:

```
return (  
  <div>  
    <b>{totalViews.data?.views} total views</b>
```

```
<div style={{ width: 512 }}>
  <h3>Daily Views</h3>
  <VictoryChart domainPadding={16}>
```

The **VictoryChart** component is a wrapper, used to combine all elements of a Victory chart. We set **domainPadding** to 16 pixels, which is a padding inside of the graph. It makes sure that the lines and bar charts do not stick to the edges of the graph, making it look slightly better.

4. Then, define a bar chart with **VictoryBar**, using **VictoryTooltip** to display the labels:

```
<VictoryBar
  labelComponent={<VictoryTooltip />}
```

The tooltip looks like this:

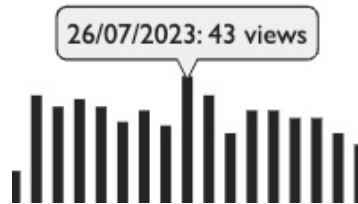


Figure 10.4 – A tooltip on a bar chart in Victory

5. Now we get to the most important part, the data. Here, we map over our **dailyViews** data returned by the query hook to bring it into a format that Victory understands:

```
data={dailyViews.data?.map((d) => ({
```

6. We map the **\_id** property to the **x-axis** value (parsing it as a date), and the **views** property to the **y-axis** value:

```
  x: new Date(d._id),
  y: d.views,
```

7. Then we create a label, where we turn the day into a local date string and then show the number of views on the given day:

```
    label: `${new Date(d._id).toLocaleDateString(): ${d.views} view
  })))}
  />
</VictoryChart>
```

```
    </div>  
  </div>  
)  
}
```

We have successfully created our first visualization using Victory! The chart will now look as follows:

### Daily Views

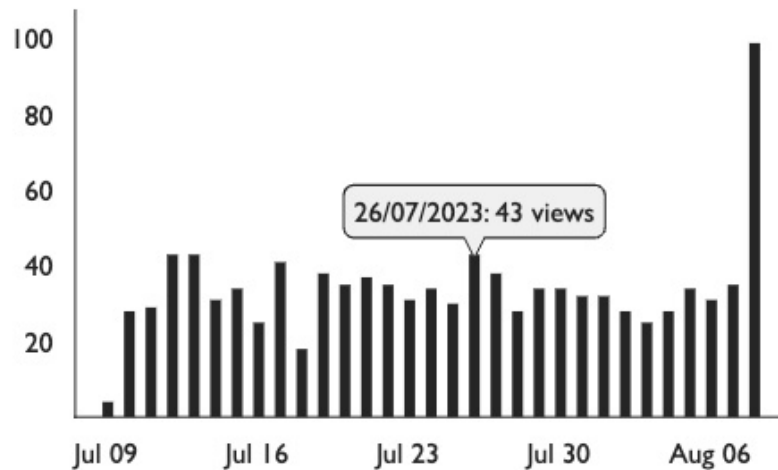


Figure 10.5 – Our first chart in Victory – a bar chart!

As you can see, Victory automatically formatted the dates for us and adjusted the axes to fit our chart into the allotted space!

Let's visualize the daily average viewing duration next.

### Creating a line chart

Creating a line chart in Victory is pretty similar to creating a bar chart, with one exception: the tooltips. In line charts, we cannot use tooltips directly, as lines could theoretically be continuous (not discrete blocks of data), so it is unclear where to place the tooltip. Instead, we use a **Voronoi container** for displaying tooltips on line charts in Victory. The name Voronoi comes from mathematics, where a Voronoi diagram partitions a region into multiple sections. In simple terms, the Voronoi container makes an intersection between the mouse position and the line chart, gets the data from that intersection point, and then displays a tooltip there.

With that in mind, let's now get started creating the line chart for the daily average viewing duration:

1. Edit **src/components/PostStats.jsx** and continue where we left off with the other chart, adding a new **VictoryChart** after the container of the bar chart:

```

    </VictoryChart>
  </div>
  <div style={{ width: 512 }}>
    <h4>Daily Average Viewing Duration</h4>
    <VictoryChart
      domainPadding={16}

```

2. In the **VictoryChart** component, we now define **containerComponent**, which will contain our **VictoryVoronoiContainer**:

```

    containerComponent={
      <VictoryVoronoiContainer
        voronoiDimension='x'

```

We defined it to only intersect with the values on the **x**-axis, meaning that the mouse pointer will only intersect with the days on our chart.

3. We can now define labels for our container, using the **datum** property to get the data entry that intersects with the mouse pointer to create a label. Our label should display the current date and the viewing duration in minutes, fixed to two decimal points:

```

    labels={({ datum }) =>
      `${datum.x.toLocaleDateString()}: ${datum.y.toFixed(2)} minute
    }

```

4. Again, we use **VictoryTooltip** to display these labels:

```

      labelComponent={<VictoryTooltip />}
    />
  }
>

```

5. Now we can finally define the **VictoryLine** chart, in which we map the data again, parsing dates and dividing the average duration to convert it from milliseconds to minutes:

```

    <VictoryLine
      data={dailyDurations.data?.map((d) => ({
        x: new Date(d._id),
        y: d.averageDuration / (60 * 1000),
      }))}
    />
  </VictoryChart>
</div>
</div>
)
}

```

As you can see, the rest was pretty simple and similar to creating the bar chart! It looks as follows:

**Daily Average Viewing Duration**



Figure 10.6 – A line chart using Victory, displaying the daily average viewing duration of a post!

As you can see, Victory is a pretty powerful library for creating charts with React, and we have only scratched the surface of what it can do! You can still customize the theme of the charts and create all sorts of complex visualizations. In this chapter, however, we focused on the most essential and widely used charts: bar and line charts.

## Summary

In this chapter, we learned about tracking events using our backend and frontend. Then, we simulated events to be used as a sample dataset for our aggregations and visualizations. Next, we learned how to aggregate data with MongoDB using the MongoDB Playground. Then, we implemented data aggregation functions in our backend. Finally, we integrated and visualized the data on the frontend using Victory.

In the next chapter, [\*\*\*Chapter 11\*\*\*](#), *Building a Backend with a GraphQL API*, we are going to learn how to use an alternative to REST, called GraphQL, to query deeply nested objects more easily.